

DOCUMENT D'ARCHITECTURE

Refonte du CRM LiVrai

Architecture cible : Spring Boot + Angular + PostgreSQL

Mars 2026 – Gourouvin Laurent

Table des matières

1. Contexte du projet.....	3
1.1 Besoins de LiVrai	3
1.2 Objectifs de refonte	3
1.3 Éléments hors périmètre	3
2. Description fonctionnelle.....	3
2.1 Les utilisateurs	3
2.2 Liste des fonctionnalités	4
2.3 Fonctionnement de l'application	4
3. Description technique	5
3.1 Stack technique.....	5
3.2 Architecture backend.....	5
3.3 Authentification — JWT.....	6
3.4 Architecture frontend	7
3.5 Communication front/back.....	7
3.6 Contrat d'API	7
3.7 Architecture globale.....	8
3.8 Conteneurisation Docker	10
4. Modèle de données	10
4.1 Ancienne structure	10
4.2 Nouvelle structure	10
4.3 Dictionnaire de données.....	10
4.4 MLD.....	12
4.5 MCD	12
5. Stratégie de migration.....	12
5.1 Données à migrer.....	12
5.2 Points de vigilance	12
5.3 Planification	13
6. Réponses aux contraintes non fonctionnelles	13
6.1 Performance.....	13
6.2 Disponibilité	13
6.3 Maintenabilité	14
6.4 Évolutivité.....	14
6.5 Volumétrie et montée en charge	14
7. Conclusion	14

1. Contexte du projet

1.1 Besoins de LiVrai

LiVrai est une entreprise de livraison B2B en forte croissance, aussi bien par le nombre de clients que par le volume de commandes. La société s'appuie sur un CRM pour gérer ses relations clients et ses livraisons.

Face à cette croissance, les équipes constatent des lenteurs, des difficultés de maintenance et s'interrogent sur la disponibilité du service lors des pics d'activité. Un audit de l'application existante a mis en évidence les limites structurelles du CRM actuel et justifié une refonte complète.

1.2 Objectifs de refonte

L'audit a identifié les problématiques suivantes qui justifient la refonte :

- Performance : absence de pool de connexions, requêtes sans pagination
- Sécurité : mots de passe en clair, absence de validation des données
- Maintenabilité : architecture MVC incomplète, logique métier dans les contrôleurs
- Évolutivité : architecture monolithique difficile à faire évoluer et à scaler
- Accès aux données : migration du pattern DAO/JDBC manuel vers Spring Data JPA

1.3 Éléments hors périmètre

La refonte se concentre sur les fonctionnalités existantes du CRM. Les éléments suivants sont hors périmètre de ce projet :

- Développement d'une application mobile (envisageable ultérieurement via l'API REST)
- Intégrations tierces (ERP, systèmes de facturation externes)
- Fonctionnalités avancées de reporting et analytics

2. Description fonctionnelle

2.1 Les utilisateurs

Dans la précédente version de l'application il y avait deux types d'utilisateurs :

Rôle	Actions
Client	Se connecter, Créer une commande, Voir ses livraisons
Administrateur	Se connecter, Créer des clients, Gérer les livraisons

Dans la nouvelle version, les clients peuvent créer leur compte de façon autonome, sans passer par le service commercial. La plateforme accueille désormais quatre types d'utilisateurs :

Rôle	Actions
Client	Créer un compte, Se connecter, Créer une commande, Voir ses livraisons, Gérer ses informations
Administrateur	Se connecter, Créer des clients, Gérer les livraisons
Service Commercial	Se connecter, Créer des clients, Gérer les livraisons, Accès à la facturation
Service Livraison	Se connecter, Gérer les livraisons, Accès à la facturation

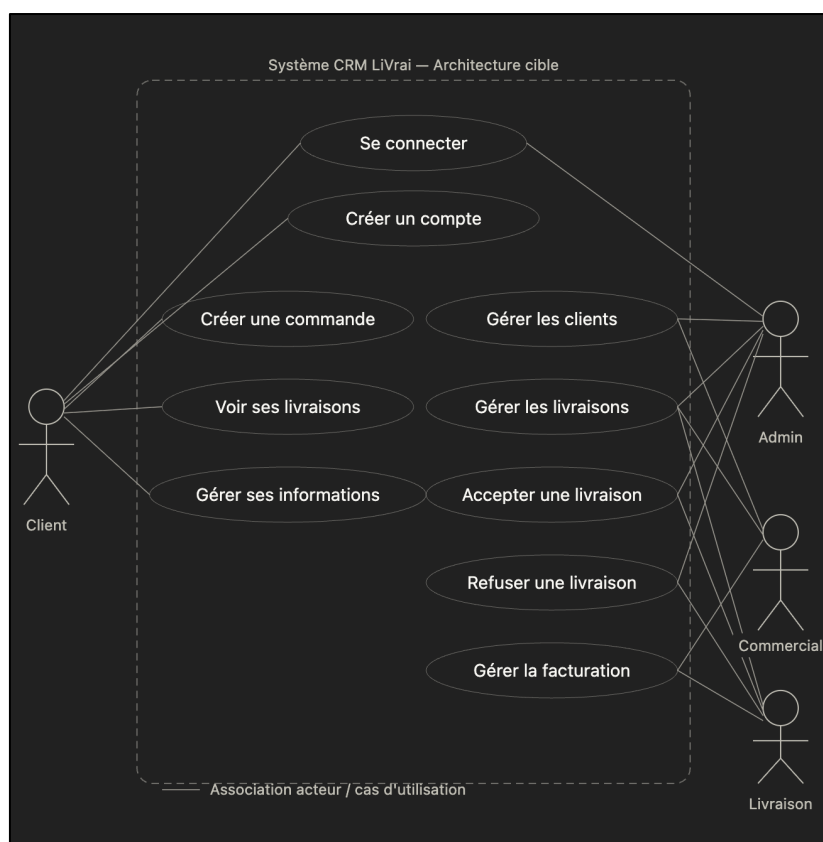
2.2 Liste des fonctionnalités

Les fonctionnalités existantes seront conservées et retravaillées. Deux nouvelles fonctionnalités sont à prévoir :

1. La gestion de la facturation, plus fine et accessible aux rôles concernés
2. La gestion des informations client (modification des coordonnées, adresses...)

Le système de rôles sera entièrement repensé pour supporter quatre niveaux d'accès distincts.

2.3 Fonctionnement de l'application



3. Description technique

3.1 Stack technique

Composant	Technologie	Version
Back-end	Java + Spring Boot (LTS)	Java 21 + Spring Boot 3.4.x
Front-end	Angular (SPA)	Angular 21 (LTS)
Base de données	PostgreSQL	v.18
Déploiement	Docker	Docker Compose

3.2 Architecture backend

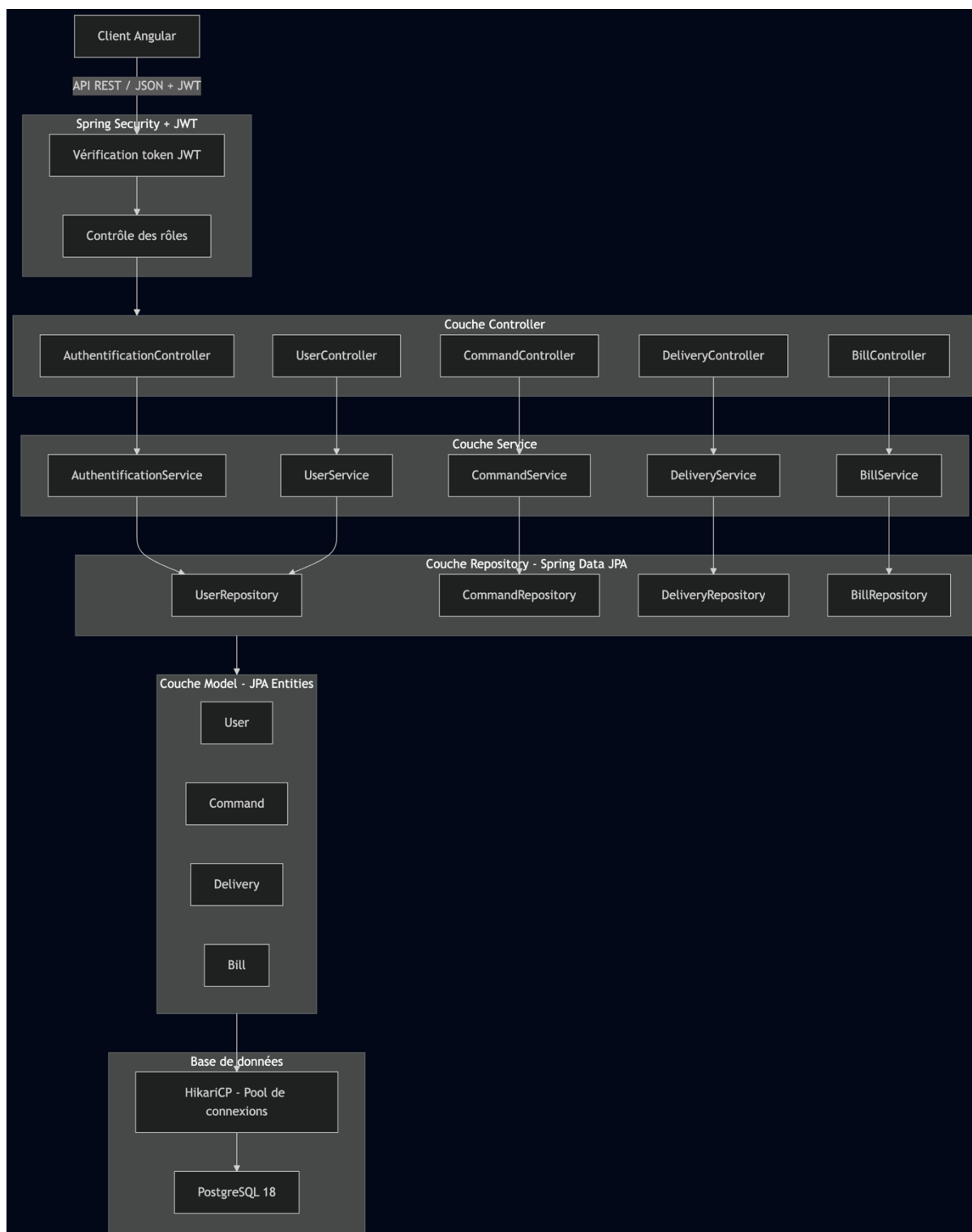
L'architecture backend repose sur Spring Boot avec une organisation en couches :

- Controller → reçoit les requêtes HTTP (remplace les Servlets)
- Service → contient la logique métier (remplace le code métier dans les Servlets)
- Repository → accès aux données via Spring Data JPA (remplace les DAO)
- Model → entités JPA mappées sur la base de données (remplace les Beans)

La sécurité est assurée par Spring Security avec authentification JWT stateless. La gestion des connexions PostgreSQL est assurée par HikariCP (pool de connexions inclus par défaut dans Spring Boot).

5 controllers et 5 services correspondants :

Controller	Service	Rôle
AuthenticationController	AuthenticationService	Connexion, génération JWT
UserController	UserService	Gestion des utilisateurs
CommandController	CommandService	Gestion des commandes
DeliveryController	DeliveryService	Gestion des livraisons
BillController	BillService	Gestion de la facturation (nouveau)



3.3 Authentification JWT

L'authentification repose sur des JSON Web Tokens (JWT). Contrairement à l'ancienne architecture à sessions serveur, JWT est stateless : le serveur génère un token signé à la connexion et le vérifie à chaque requête sans stocker d'état. Cette approche permet la scalabilité horizontale et s'intègre nativement avec Spring Security et Angular.

3.4 Architecture frontend

Le frontend Angular 21 adopte une architecture Standalone Feature-based : découpage métier plutôt que technique. Chaque feature regroupe ses composants, services et modèles.

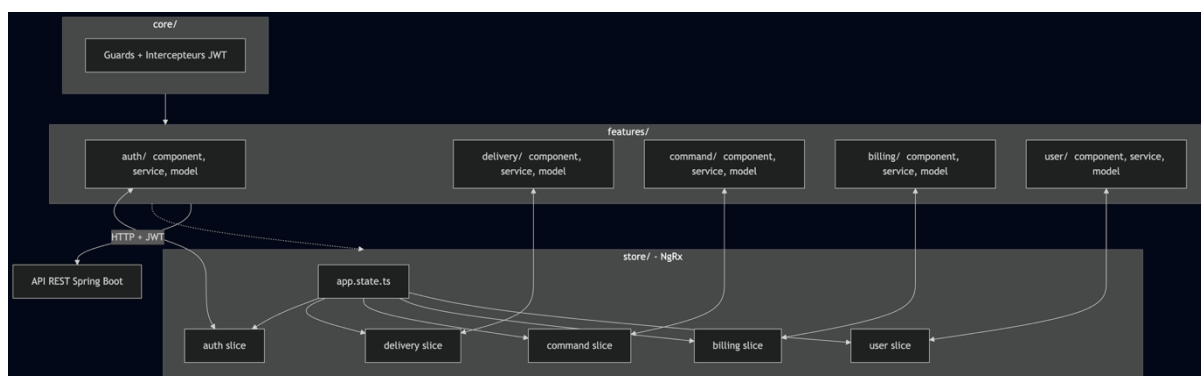
Structure de l'application :

```

app/
  core/          -> services globaux, guards, intercepteurs JWT
  store/         -> store global NgRx
  app.state.ts   -> interface globale
  features/
    auth/        -> connexion, inscription
    delivery/    -> liste et suivi des livraisons
    command/     -> création de commande
    billing/     -> facturation
    user/        -> gestion du profil client

```

La gestion d'état est assurée par NgRx (store centralisé). Chaque feature dispose de son propre slice (actions, reducer, selectors, effects). Les effects appellent les services HTTP : jamais les composants directement.



3.5 Communication front/back

Le frontend consomme l'API REST du backend via le module HttpClient d'Angular. Les échanges se font en JSON. Un intercepteur HTTP (AuthInterceptor) ajoute automatiquement le token JWT dans le header Authorization de chaque requête.

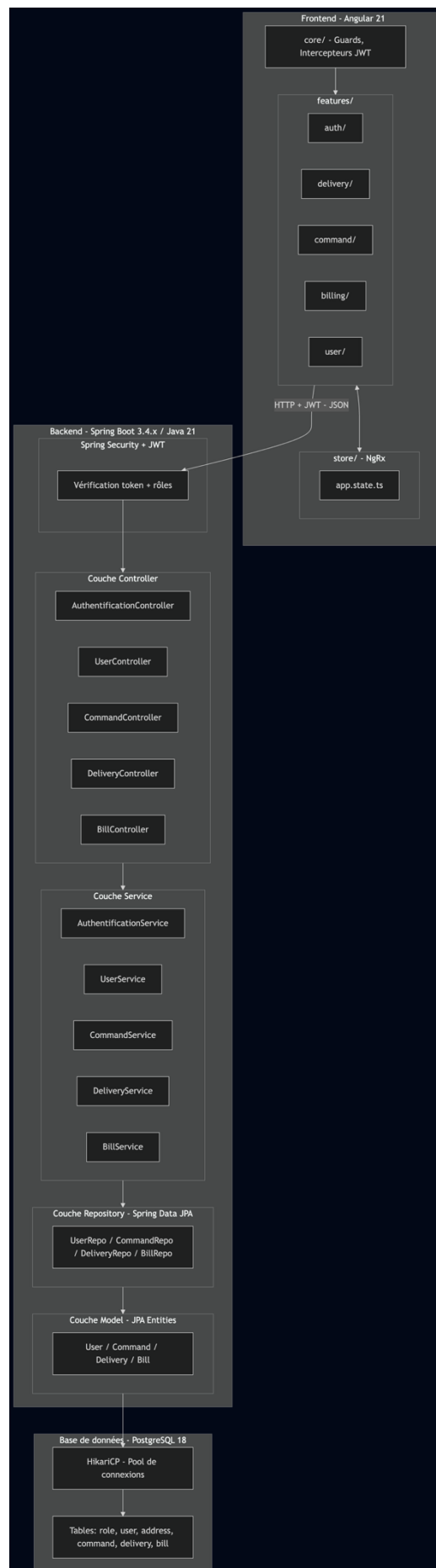
Les routes Angular sont protégées par des Guards (AuthGuard, RoleGuard). Spring Security vérifie le token JWT et le rôle à chaque requête entrante : la sécurité est assurée aux deux niveaux.

3.6 Contrat d'API

Endpoint	Méthode	Rôle requis	Description
/api/auth/login	POST	Public	Connexion et génération du token JWT
/api/auth/register	POST	Public	Création de compte client
/api/users	GET	ADMIN	Liste tous les utilisateurs

/api/users/{id}	GET	ADMIN, COMMERCIAL	Détail d'un utilisateur
/api/users/{id}	PUT	ADMIN, CLIENT	Modification des informations
/api/users/{id}	DELETE	ADMIN	Suppression d'un utilisateur
/api/commands	GET	ADMIN, COMMERCIAL	Liste toutes les commandes
/api/commands	POST	CLIENT	Crée une nouvelle commande
/api/commands/{id}	GET	ADMIN, COMMERCIAL, CLIENT	Détail d'une commande
/api/deliveries	GET	ADMIN, COMMERCIAL, LIVRAISON	Liste toutes les livraisons
/api/deliveries/{id}	GET	Tous les rôles	Détail d'une livraison
/api/deliveries/{id}/accept	PATCH	ADMIN, LIVRAISON	Accepte une livraison
/api/deliveries/{id}/reject	PATCH	ADMIN, LIVRAISON	Refuse une livraison
/api/bills	GET	ADMIN, COMMERCIAL, LIVRAISON	Liste toutes les factures
/api/bills/{id}	GET	ADMIN, COMMERCIAL, LIVRAISON	Détail d'une facture
/api/bills	POST	ADMIN, COMMERCIAL	Crée une facture

3.7 Architecture globale



3.8 Conteneurisation Docker

L'application est conçue Docker-ready. Trois services sont définis :

Service	Image	Port exposé	Rôle
livrai-backend	Java 21 + Spring Boot	8080	API REST
livrai-frontend	Node + Angular + Nginx	80	SPA Angular
livrai-db	postgres:18	5432 (interne)	Base de données

Les trois services communiquent via un réseau Docker interne. La base de données n'est pas exposée à l'extérieur. Cette architecture permet de scaler horizontalement le backend sans toucher au frontend ni à la base.

4. Modèle de données

4.1 Ancienne structure

La base de données existante (MySQL 8) se compose de seulement deux tables : user et delivery. Cette structure minimale ne peut pas supporter les nouveaux besoins fonctionnels :

- Le champ booléen admin est incompatible avec un système de 4 rôles
- Pas de séparation entre commande et livraison
- Pas de table facturation
- Mots de passe en clair
- Absence d'index explicites

4.2 Nouvelle structure

La base de données sera migrée de MySQL 8 vers PostgreSQL 18. La nouvelle structure s'articule autour de 6 tables :

- role : gestion des 4 niveaux d'accès
- user : utilisateurs avec référence vers role (mot de passe hashé bcrypt)
- address : adresses de livraison liées aux utilisateurs
- command : commandes passées par les clients
- delivery : livraisons associées aux commandes (statut contraint par ENUM)
- bill : factures liées aux livraisons (nouveau)

4.3 Dictionnaire de données

Attribut	Entité	Type	Contraintes	Description
id	Role	SERIAL	PK, NOT NULL	Identifiant unique
name	Role	VARCHAR(64)	NOT NULL, UNIQUE	CLIENT, ADMIN, COMMERCIAL, LIVRAISON

created_at / updated_at	Role	TIMESTAMP	NOT NULL	Dates de création et modification
id	User	SERIAL	PK, NOT NULL	Identifiant unique
email	User	VARCHAR(255)	NOT NULL, UNIQUE	Adresse email de connexion
name	User	VARCHAR(128)	NOT NULL	Nom complet
password	User	VARCHAR(255)	NOT NULL	Mot de passe hashé (bcrypt)
phone	User	VARCHAR(20)	NULLABLE	Numéro de téléphone
role_id	User	INT	FK → Role, DEFAULT 2	Rôle (CLIENT par défaut)
created_at / updated_at	User	TIMESTAMP	NOT NULL	Dates de création et modification
id	Address	SERIAL	PK, NOT NULL	Identifiant unique
street / city / zip_code / country	Address	VARCHAR	NOT NULL	Composantes de l'adresse
user_id	Address	INT	FK → User, NOT NULL	Propriétaire de l'adresse
id	Command	SERIAL	PK, NOT NULL	Identifiant unique
volume	Command	INT	NOT NULL, > 0	Volume en m³
weight	Command	INT	NOT NULL, > 0	Poids en kg
user_id / address_id	Command	INT	FK, NOT NULL	Client et adresse de livraison
id	Delivery	SERIAL	PK, NOT NULL	Identifiant unique
status	Delivery	ENUM	NOT NULL	PENDING, ACCEPTED, REJECTED, DONE
scheduled_at	Delivery	TIMESTAMP	NULLABLE	Date prévue de livraison
command_id	Delivery	INT	FK, NOT NULL, UNIQUE	Commande associée (1-1)
id	Bill	SERIAL	PK, NOT NULL	Identifiant unique
amount	Bill	DECIMAL(10,2)	NOT NULL, > 0	Montant de la facture

delivery_id	Bill	INT	FK, NOT NULL, UNIQUE	Livraison facturée (1-1)
-------------	------	-----	-------------------------	-----------------------------

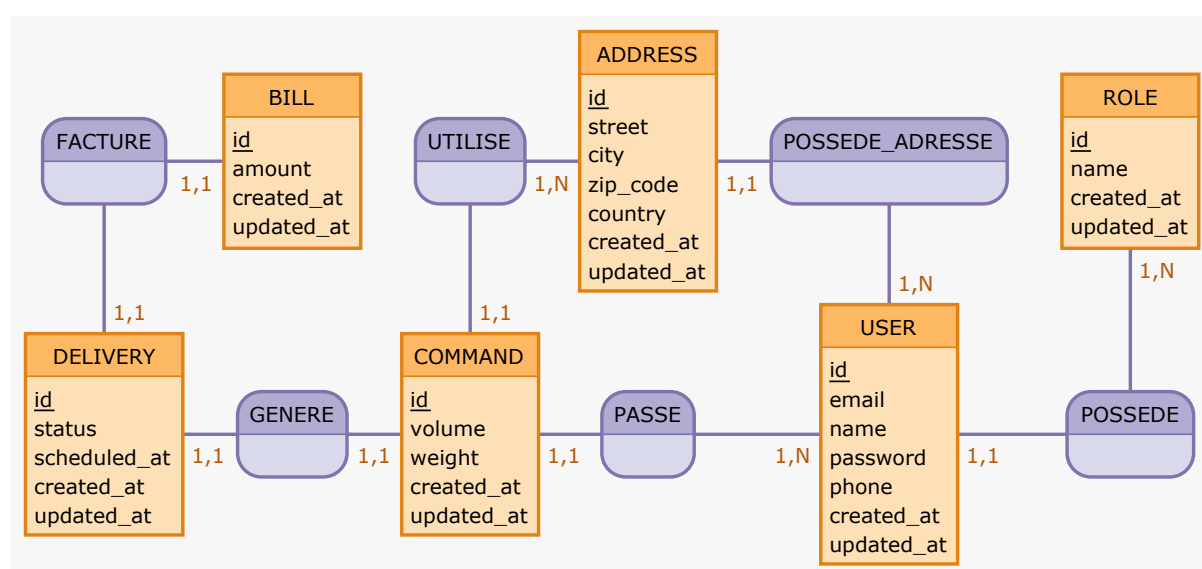
4.4 MLD

```

ROLE (id, name, created_at, updated_at)
USER (id, email, name, password, phone, created_at, updated_at, #role_id)
ADDRESS (id, street, city, zip_code, country, created_at, updated_at, #user_id)
COMMAND (id, volume, weight, created_at, updated_at, #user_id, #address_id)
DELIVERY (id, status, scheduled_at, created_at, updated_at, #command_id)
BILL (id, amount, created_at, updated_at, #delivery_id)

```

4.5 MCD



5. Stratégie de migration

5.1 Données à migrer

Ancienne table	Nouvelle(s) table(s)	Transformation nécessaire
user	role + user	Convertir le booléen admin en role_id
delivery	command + delivery	Créer une command pour chaque delivery existante

5.2 Points de vigilance

Mots de passe en clair

Les mots de passe de l'ancienne base sont stockés en clair. Lors de la migration, il suffira de les hasher avec bcrypt et de les stocker dans la nouvelle base. Aucune intervention des utilisateurs n'est nécessaire.

Conversion des rôles

Le champ admin booléen doit être converti en role_id :

- admin = TRUE → rôle ADMIN
- admin = FALSE → rôle CLIENT

Les nouveaux rôles SERVICE_COMMERCIAL et SERVICE_LIVRAISON seront attribués manuellement par l'administrateur après la migration.

Séparation commande / livraison

L'ancienne table delivery joue les deux rôles. Pour chaque enregistrement existant, il faudra créer une entrée dans command et une entrée dans delivery liées entre elles.

5.3 Planification

3. Vendredi soir : mise en maintenance de l'application
4. Export des données MySQL via mysqldump
5. Création de la nouvelle structure PostgreSQL (init-schema.sql)
6. Transformation et import des données (hashage bcrypt, conversion des rôles, séparation commande/livraison)
7. Tests de validation : vérification de l'intégrité des données
8. Déploiement de la nouvelle application
9. Dimanche soir : remise en production

 *Le script de migration détaillé sera produit lors de la phase d'implémentation et testé sur un environnement de recette avant toute exécution en production.*

6. Réponses aux contraintes non fonctionnelles

6.1 Performance

Contrainte	Réponse architecturale
Connexions multiples à la BDD	HikariCP : pool de connexions inclus par défaut dans Spring Boot
Requêtes sans pagination	Pagination obligatoire sur tous les endpoints de liste via Spring Data JPA
Absence d'index	Index explicites sur email, role_id, user_id, status dans PostgreSQL
Rendu serveur JSP lent	Angular SPA : rendu côté client, seules les données transitent via l'API

6.2 Disponibilité

Contrainte	Réponse architecturale
Sessions serveur bloquantes	JWT stateless : aucun état stocké côté serveur
Architecture monolithique	Séparation front/back : les deux peuvent être redémarrés indépendamment

Pics d'activité	Architecture Docker-ready : scalabilité horizontale possible
-----------------	--

6.3 Maintenabilité

Contrainte	Réponse architecturale
Logique métier dans les contrôleurs	Architecture en couches : Controller / Service / Repository / Model séparés
Mélange présentation/métier	Séparation front/back : Angular gère la présentation, Spring Boot la logique
Code verbeux (DAO manuels)	Spring Data JPA : repositories générés automatiquement

6.4 Évolutivité

Contrainte	Réponse architecturale
Architecture monolithique JSP/Servlet	API REST : tout nouveau client peut consommer l'API sans modification du backend
Système de rôles limité (booléen)	Table role dédiée : ajout d'un nouveau rôle sans modifier la structure
Stack obsolète	Spring Boot 3.4.x + Angular 21 : stack moderne avec support LTS garanti
Déploiement manuel	Docker : conteneurisation facilitant les déploiements

6.5 Volumétrie et montée en charge

Contrainte	Réponse architecturale
Absence de pagination	Pagination systématique sur les endpoints de liste
Absence d'index	Index sur les colonnes fréquemment interrogées
MySQL limitations à grande échelle	PostgreSQL 18 : meilleure gestion des transactions et des grandes volumétries
Deux tables pour tout gérer	Schéma normalisé en 6 tables : meilleure performance des requêtes ciblées

7. Conclusion

Ce document d'architecture définit la cible technique de la refonte du CRM LiVrai, en s'appuyant directement sur les constats de l'audit réalisé en première phase.

La nouvelle architecture répond aux quatre enjeux identifiés :

- Performance : HikariCP, pagination et index PostgreSQL éliminent les goulots d'étranglement identifiés.
- Disponibilité : l'authentification JWT stateless et l'architecture Docker-ready permettent une scalabilité horizontale impossible avec l'ancienne architecture à sessions.
- Maintenabilité : la séparation claire des responsabilités côté backend et l'architecture Feature-based Angular rendent le code plus lisible, testable et évolutif.
- Évolutivité : l'API REST ouvre la plateforme à de futurs clients sans modification du backend. La table role dédiée permet d'ajouter de nouveaux profils sans restructurer la base.

La migration de MySQL vers PostgreSQL, planifiée sur un weekend, permettra de conserver l'intégralité des données existantes tout en bénéficiant des améliorations structurelles du nouveau schéma.

Ce document constitue la base de référence pour la phase d'implémentation. Le script de migration détaillé, les Dockerfiles et la configuration Spring Security feront l'objet de livrables séparés lors du démarrage du développement.